TD6: Programmation dynamique et algorithmes gloutons

CPES ≪ Sciences des données, arts et cultures ≫

Antoine GAUQUIER
antoine.gauquier@ens.fr

Pierre ABOULKER
pierreaboulker@gmail.com

14 janvier 2025

Dans ce TD, il vous est proposé de résoudre deux exercices types TD afin de vous préparer à l'examen : un sur la programmation dynamique et un sur les algorithmes gloutons.

Empilement de tuiles

On souhaite former des *tours* en utilisant des tuiles de largeur différente. On décrit n la taille d'une tour, à savoir le nombre de tuiles empilées qui la constitue. m décrit la largeur maximale possible pour une tuile. Ainsi, si $m = 3$, alors nous avons trois largeurs de tuiles possible : 1, 2 et 3. Enfin, k décrit le nombre de tuiles d'une même largeur que l'on peut utiliser pour une tour.

L'objectif est alors de calculer, étant donné un triplet (n, m, k), le nombre de tours différentes qu'il est possible de réaliser. Deux points importants sont à noter :

— Les tours doivent être *stables* : chaque tuile d'une tour doit donc reposer sur une tuile de largeur supérieure ou égale à la sienne.

— Deux tours sont considérées différentes si il existe au moins une hauteur h ($h \in \{1, \dots, n\}$) à laquelle la largeur de la tuile de la première tour est différente de celle de la seconde.

Par exemple, si on a $(n, m, k) = (3, 3, 2)$, alors il est possible de réaliser 7 tours différentes : [1, 1, 2], [1, 1, 3], [1, 2, 2], [1, 2, 3], [1, 3, 3], [2, 2, 3] et [2, 3, 3] ; où chaque liste représente les tuiles d'une tour (le bas de la tour étant à la fin du tableau).

Question 1 Justifier pourquoi l'utilisation de la programmation dynamique est pertinente dans ce contexte.

Question 2 Proposer une stratégie de résolution du problème basée sur la récursivité, en donnant une relation récursive du nombre de tours différentes possibles en fonction des trois paramètres (n, m, k). Donner également les cas de base (d'arrêt) nécessaires.

Question 3 Implémenter un algorithme permettant de résoudre le problème, en exploitant la question précédente. Proposer une version avec mémoïsation, et une version sans.

Question 4 Quelle sont les complexités temporelles et spatiales de l'algorithme sans mémoïsation ? Justifier. La complexité temporelle peut-elle être différente dans la version avec mémoïsation ?

Arrêts de bus

Une ville souhaite raccorder un nouveau lotissement à son réseau de transports en commun. Pour ce faire, elle souhaite construire un certain nombre d'arrêts de bus, à partir desquels elle complètera les lignes déjà existantes.

Afin de couvrir équitablement l'ensemble du lotissement, la ville se fixe la règle suivante : chaque maison doit être distante d'au plus ℓ mètres d'un arrêt.

Pour simplifier la résolution, on va considérer que la ville procède rue par rue, et on se propose de traiter le problème pour une de ces rues. On formalise alors ce dernier de la façon suivante :

- La rue est composée de n maisons différentes.
- Chaque maison est représentée par sa distance au début de la rue, en mètres. On note x_i cette distance, pour tout $i \in [\![1, n]\!]$.
- Les maisons sont ordonnées selon l'ordre croissant des distances. Cela implique donc que $x_i \leqslant x_j$ si $i < j$.
- L'objectif de la ville est de déterminer le nombre minimal d'arrêts à créer pour desservir l'ensemble des maisons de la rue, sachant que pour rappel, une maison est considérée desservie si elle se trouve à moins de (ou à exactement) ℓ mètres d'un arrêt. Elle souhaite également obtenir la position de ces arrêts dans la rue.
- **On considère que les maisons ainsi que les arrêts sont sur une même droite**, donc les distances ne sont calculées que sur une dimension.

Question 5 Afin de bien comprendre la formalisation, on pose $\ell = 100$ **uniquement pour cette question**, et on considère une rue avec $n = 5$ maisons, dont les positions sont $(x_1, x_2, x_3, x_4, x_5) = (180, 200, 370, 390, 590)$. Faire un dessin et déterminer la solution optimale dans cette configuration (le nombre d'arrêts à créer **et** la position de ceux-ci).

Question 6 Proposer un critère de gloutonnerie pour déterminer la position des arrêts. *Indication : Commencez par réfléchir à la position du premier arrêt de la rue, puis tentez ensuite de généraliser.*

Question 7 Écrire le pseudo-code d'un algorithme suivant cette stratégie gloutonne qui résoud le problème.

Question 8 Quelle est la complexité temporelle de cet algorithme ? Justifier. Comment serait impactée la complexité si maintenant les maisons n'étaient pas triées par ordre croissant de distance au début de la rue ?

Question 9 Démontrer que la stratégie gloutonne choisie mène à une solution optimale. *Faites valider votre stratégie avant de vous lancer dans une preuve si vous commencez cette question avant la correction de la question 6.*